Project: CI/CD for Cloud Native

Due: 04/04/2026 11:59 PM
Extended Deadline: 04/07/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Tekton CI Pipeline
- Task Overview
- Task

Flux CD Pipeline
- Task Overview
- Task

Wrap Up
- Conclusion

Tekton CI Pipeline (Task)

Continuous Integration (CI)

In Unit 1, you built a CI/CD pipeline that:

- Runs on the GitHub Actions platform
- Executes test cases whenever a developer pushes to the `master` branch
- Builds your application into a container image
- Publishes the image to Azure Container Registry

GitHub Actions is widely used in the DevOps ecosystem. In this unit, however, you will learn how to achieve the same outcomes using a different toolchain: running tests, publishing container images to a central registry, and deploying both your LLM service and Login service to your Kubernetes cluster.

As you have seen so far, your cluster is currently underutilized, hosting only two microservices. In real-world environments, clusters handle far more than production workloads. They also run operational tasks such as **CI/CD pipelines**, batch jobs, data processing, and automation workflows.

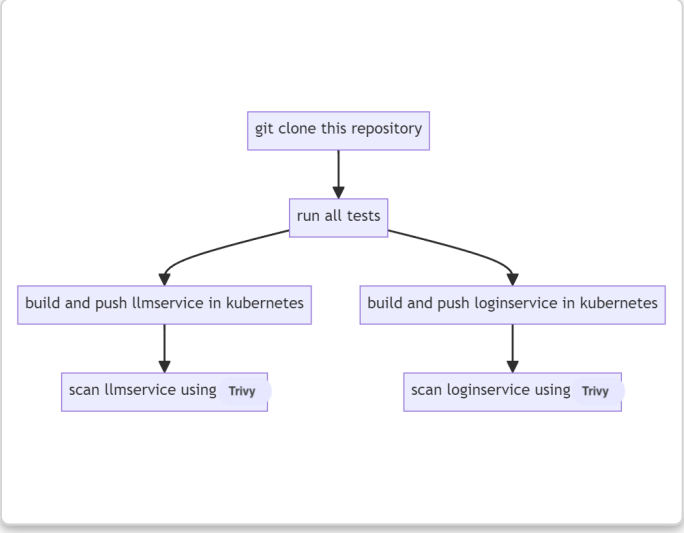
Moreover, building container images often requires access to private resources like internal registries and
credentials. When CI/CD jobs run outside the cluster, we compromise security by relying on external systems. In such
cases, it is possible for a supply-chain attack against widely used dependencies to occur without our knowledge. By controlling the build environment within our cluster, we can verify exactly how and where our images were built.

Over the years, **Tekton** has emerged as a leading solution in the cloud-native CI/CD landscape. Unlike hosted CI systems, Tekton runs *inside your Kubernetes cluster*. This represents a fundamental shift: your cluster is not just a place to run applications — it can also execute the processes that build, test, and deploy them.

Tekton listens for CI/CD events, dynamically provisions pods to execute pipeline steps, and scales those workloads using the same Kubernetes mechanisms that power your applications. It also provides a dashboard for monitoring pipeline runs, similar to the visibility you get from GitHub Actions.

By the end of this task, you will understand how cloud-native teams implement CI/CD in a Kubernetes-centric way — bringing the entire software delivery lifecycle into the cluster itself.

Here's a high-level diagram showing how your pipeline will look like at the end of this task:



What Each Subtask Teaches

Subtask 1 (Setup): Setting things up.

Subtask 2 (Tekton Install): Learn Tekton's core concepts (Tasks, Pipelines, Workspaces).

Subtask 3 (Get Tasks): Pull and inspect reusable Tekton Tasks (`git-clone`, `kaniko`).

Subtask 4 (Pipeline v1): Build a basic clone → build → push pipeline for the LLM service.

Subtask 5 (Pipeline v2): Extend the pipeline to build the Django/login service.

Subtask 6 (Security Scan): Scan images with Trivy and handle private registry authentication.
Subtask 7 (Testing Gate): Add a test task that blocks builds when tests fail.

Prerequisites

Before starting this task, confirm you have the following tools installed on your local machine:

- Tekton CLI (`tkn`) — for monitoring pipeline runs (installed in Subtask 2)
- `jq` — used when generating ACR credentials
- `base64` — available by default on Linux/macOS;

Subtask 1: Setup

The previous project on productionizing your LLM server introduced you to building a basic CI pipeline using GitHub Actions. While this works well for non-cloud-native projects, we want to take things a step further by using Tekton to manage our Continuous Integration pipeline.

1. As a first step, modify your GitHub Actions pipeline in `.github/workflows/main.yml` so that it no longer triggers automatically. Currently, the pipeline runs on every push to the main branch. We won't walk you through this — you've built a whole distributed system at this point, so disabling a pipeline trigger should be straightforward.
2. The `handout` section contains instructions on how to download the files needed for this project. Make sure to download them **outside** your project folder.

Yet Another Refactoring

Our app has been working well across the last few projects, but there's an anti-pattern we've been carrying along. It's easy to get away with in small projects, but it doesn't hold up at production scale.

Here's what we've been doing:

1. The LLM server logic is written in Python using FastAPI.
2. When containerizing the app, we copied the model file — `tinyllama-1.1b-chat-v1.0.Q2_K.gguf` (~500MB) — directly into the container image. This works, but it's not the right approach.
3. At production scale, model files are typically several gigabytes. Baking them into the image makes the container unnecessarily large, slows down image pulls, and increases build times. You may have already noticed this — your CI/CD pipeline from Unit 1 took a long time to build for exactly this reason.
4. The right principle is simple: **a container image should only contain application logic**. Any data the application needs should be provided externally at runtime.

To accomplish this, we use the following Kubernetes primitives:

PersistentVolume (PV) & PersistentVolumeClaim (PVC): provide a storage location in the cluster that exists independently of any pod. The model file lives here, not in the image.

Init containers: run before the main application container starts. We use one to download the model file onto the PersistentVolume. Crucially, it checks whether the file already exists before downloading — so the download only ever happens once. Any subsequent pod restart or scale-out will find the model already on the volume and skip straight to starting the app.

Environment variables: the path to the model file is passed into the container as an environment variable (`MODEL_PATH`), keeping the application code decoupled from any specific file system layout.

Inspect the handout files

1. Open the handout files in your VS Code editor.
2. You should see a folder structure like this:

```
cicd-llm-app-handout
├── deployment.yaml
├── Dockerfile
├── main.py
├── Makefile
├── model-pvc.yaml
└── service.yaml
```

1. Spend about 5 minutes reading the contents in the above files. We've added comments explaining what each file does.
2. Replace your main.py file in the **inner llm_server** directory with the one you just downloaded. The only change made was how the model path is passed. The app will attempt to read the model path from the environment variable. If it's not present, it will default to the local path. The local path is what's used during development.
3. Replace the `Dockerfile` in **llm_server/docker/Dockerfile** with the new Dockerfile. The major change here is to **NOT** copy the model file into the container image.
4. `model-pvc.yaml` is a new file in this project. It tells Kubernetes that the app needs storage. Kubernetes then instructs Azure to provision an Azure Files share to satisfy that request. Once the storage is created, it stays around regardless of what happens to your pods. You should put this file in the **llm_server/k8s** directory.
5. Replace your deployment.yaml file in `llm_server/k8s` with the downloaded deployment.yaml file. This has grown from what you've used previously. The biggest addition is an **init container** — a setup step that runs before your app starts and downloads the model file onto the **PersistentVolume**. If the file is already there, it skips the download. Only once that's done does Kubernetes start your actual app. Both the init container and the app container mount the same volume at **/models**, which is how the downloaded file gets passed between them. The model path is then handed to the app

via an environment variable **MODEL_PATH**, so the app knows exactly where to find it.

6. Finally, there is a **Makefile** in the `llm_server` directory with commands for building the container image, testing it locally, and deploying it to the cluster. Also update your `service.yaml` to use a `LoadBalancer` instead of `ClusterIP` — this gives the service a public IP so you can call it directly from your browser or terminal during testing.

Build and Deploy your LLM Service

Start by rebuilding the wheel file for the LLM service. There is a command in the **Makefile** for this.

Next, build the container image and test it locally using the following command (also available in the Makefile):

```
docker run -it -p 8000:8000 \
  -v ./tinyllama-1.1b-chat-v1.0.Q2_K.gg
  -e MODEL_PATH=/models/tinyllama-1.1b-
  llmapp:latest
```

This mounts the model file from your local machine into the container and passes its path as an environment variable — the same way it works in the cluster.

While the container is running, open a new terminal and send a request to it:

```
curl -G --data-urlencode 'message=what
```

You should get a response that looks something like this:

```
{"object":"text_completion","created":1
```

Pay attention to the `model` field. It should start with `/models/...`. If it doesn't, the environment variable is not being picked up correctly — do not proceed until this is resolved.

Once local testing passes, push the image to ACR using the command in the Makefile, then deploy your Kubernetes manifests in this order:

1. `model-pvc.yaml`
2. `deployment.yaml`
3. `service.yaml`

You should be able to make requests to your LLM service via the LoadBalancer IP. Make sure that the llm response contains the new model path, **models/....**. Do NOT proceed to other sub-tasks without getting this right.

Cluster State

Your cluster state should look like this:

```
azureuser@qa-vm:~/cloud-native-handout-
NAME                                  RE
llmapp-deployment-6d64847f7c-2pgnb    1/

azureuser@qa-vm:~/cloud-native-handout-
NAME            STATUS    VOLUME
model-storage   Bound     pvc-fc626e9d-2

azureuser@qa-vm:~/cloud-native-handout-
NAME
pvc-fc626e9d-20cf-4ae4-b2bb-94747396110

azureuser@qa-vm:~/cloud-native-handout-
NAME             TYPE           CLUSTER
kubernetes       ClusterIP      10.0.0.
llmapp-service   LoadBalancer   10.0.21
```

If you get stuck, post on the discussion forum.

Subtask 2: A Task of Tektonic Proportions

To work with Tekton, you need to install two things: the Tekton Pipelines controller (which runs on the cluster) and the
Tekton CLI (which runs on your development VM).

Install the pipeline components with the following commands.

```
kubectl apply --filename https://storag
```

Information

This command downloads and applies Tekton's official installation manifest — a large YAML file containing all the Kubernetes resources required to run Tekton Pipelines.

Key points:

• **Dedicated namespace:** Tekton system components are installed into the `tekton-pipelines` namespace (created automatically if it does not already exist).

• **Control-plane components:** The manifest deploys controllers, webhooks, service accounts, RBAC permissions, and configuration objects that enable Tekton to orchestrate CI/CD workflows inside your cluster.

Install Tekton CLI

Run the following command to install Tekton CLI:

```
# Get the tar.xz
curl -LO https://github.com/tektoncd/cl
# Extract tkn to your PATH (e.g. /usr/l
sudo tar xvzf tkn_0.43.0_Linux_x86_64.t
```

The Tekton CLI (tkn) provides a convenient way to interact with Tekton resources from the command line. While you can use kubectl alone, many Tekton operations — such as viewing pipeline runs, streaming logs, inspecting tasks, and triggering executions — are much easier and more intuitive with the dedicated CLI.

You will use tkn to:

Monitor pipeline and task runs in real time
View logs for individual steps without manually locating pods
Inspect Tekton resources in a human-readable format

Troubleshoot failed CI/CD executions more efficiently

Now that you've setup Tekton, you should run the following command to observe the Tekton resources in your cluster.
We'll start with the tekton pods:

```
kubectl get pods --namespace tekton-pip
```

You should see an output similar to what we have below:

```
tekton-events-controller-bcd5b75f7-tt5f
tekton-pipelines-controller-868956fb6c-
tekton-pipelines-webhook-b974bd4b-n7cj5
```

If all the pods are `READY`, then we can start building our CI pipeline!

Information

You may have noticed we had to manually log in to the AKS cluster for `kubectl` to work.
With all pipelines, there are ways to automate this step, but nothing that is widely established as a best practice.

Tekton's Basics

To understand Tekton, let's think about what we would want from a CI pipeline built in Kubernetes:

1. Get access to a GitHub repository.
2. Get access to the ACR we want to build the image on.
3. A set of tasks we want to perform.
4. The order to perform them in, arranged so that we can be parallel if we are able to.
5. A way to share information between tasks.

Given these considerations, Tekton provides easy ways to set up:

1. Kubernetes and GitHub integration.
2. Task definitions, to state what exactly we are running at any point as a "task".

3. Pipeline definitions, which state what steps to run and in what order.
4. Workspaces, which let us share files and credentials between tasks and input information.

In a lot of ways, it pays to look at Tekton as a layer on top of standard Kubernetes concepts:

1. `Task`: Defines a set of steps to run. It creates a Pod.
2. `Pipeline`: Defines the order of execution. It manages the dependency graph (e.g., "Don't run Build until Test passes").
3. `TaskRun / PipelineRun`: These are the execution records.
   Think of a Task as a function definition (`def build_image():`), and a TaskRun as the function call (`build_image(version=1.0)`).
4. `Workspaces`: These act like `Persistent Volumes` that are automatically bound and mounted for the duration of the pipeline, allowing you to pass files (like source code) between tasks.

Given these similarities, why not just use standard Kubernetes Jobs?

You could, but you would face a major headache: **Orchestration**. Kubernetes Jobs do not natively know how to wait for
each other. If you wanted Job B to run only after Job A succeeded, you would have to write a custom script to poll the
API and check the status.

Tekton handles this ordering logic automatically. It also manages the lifecycle of shared storage (Workspaces),
cleaning them up when the pipeline finishes so you don't waste resources.
Finally, the `tkn` CLI provides a developer-friendly view of these processes, which is much easier to read than sifting
through raw Pod logs.

Install the Tekton Dashboard

As future cloud practitioners, it is important to be able to visualize and monitor the state of your CI/CD workflows. While command-line tools are powerful, dashboards provide an intuitive interface for observing pipeline activity, tracking progress, and diagnosing failures.

Tekton provides a web-based Dashboard that allows you to:

• View pipelines, tasks, and pipeline runs
• Monitor execution status in real time
• Inspect logs and failure details
• Understand the overall health of your CI/CD workflows

In this task, you will install the Tekton Dashboard and use it to observe your pipeline executions.

Follow the official installation instructions here:

https://tekton.dev/docs/dashboard/install/

Cluster State

After installing the dashboard, your cluster state should look like this:

```
kubectl get pods --namespace tekton-pipe

tekton-dashboard-6d844bf975-r8645
tekton-events-controller-bcd5b75f7-tt5fl
tekton-pipelines-controller-868956fb6c-2
tekton-pipelines-webhook-b974bd4b-n7cj5
```

The dashboard works relatively well, but it can fail from time to time. You are free to visualize the cluster however you want to for this task.

Viewing your Tekton Dashboard

For those using VS Code's Remote SSH extension to connect to your VM, you can access the Tekton Dashboard by forwarding the dashboard service port to your local machine:

1. Open a new terminal in VS Code and run:

```
kubectl --namespace tekton-pipelines
```

This forwards the dashboard service running inside your cluster to port **9097** on your local computer.

2. Use SSH tunneling directly from your local machine. Run the following command in your local terminal:

```
ssh -L 9097:localhost:9097 <your-vm-use
```

1. Then open http://localhost:9097 in your local browser.

You should now be able to view and interact with your pipeline runs through the web interface.

Getting Authenticated

With all that said, we now need to talk about authenticating Tekton to the two parts of our CI pipeline that are external to the cluster:

Our GitHub repository
Our Azure Container Registry (ACR)

Starting with the GitHub repository, our first goal will be to configure our repository to work with Tekton. As Tekton, and your AKS, is an external resource compared to Azure DevOps and your GitHub Actions, we need a token for this connection.

In previous tasks, you likely used a Personal Access Token (PAT). While easy to set up, PATs are tied to your user account. If you leave the organization or your account is compromised, the pipeline breaks or the attacker gains your full permissions.

Instead, we will use a GitHub Deploy Key. This is an SSH key paired specifically to one repository. It grants the CI system access only to that specific codebase, adhering to the principle of least privilege.

To start, run the following to generate your key pair, **specifying the password to be nothing**.

```
ssh-keygen -t rsa -b 4096 -m PEM -f pat
```

Place your key file **outside of any git repositories**, as we do **not** want this key to accidentally get committed anywhere.

This will create two files at the specified path:

`keyname`: the private key,
`keyname.pub`: the public key.

Now that you have that information, do the following to add your deploy key:

1. Go to the GitHub page for your repository.
2. Go to your repository's "Settings" page, which is located next to the "Insights" section.
3. In the sidebar, click **Deploy Keys**.
4. Click **Add deploy key**.
5. Paste in the contents of `keyname.pub` into the `Key` field.
6. Leave "Allow write access" unchecked.
7. Click **Add Key**.

With that done, you now have an SSH key that can be used by Tekton to authenticate with your Git repository. However, Tekton does not consume SSH keys directly. Instead, authentication is handled by mounting Kubernetes Secrets into Tekton Tasks and Workspaces.

To enable Git authentication, we must create a Kubernetes Secret containing the SSH private key and annotate it so that Tekton's git-clone task can recognize and use it.

For Tekton, the format is the following:

```
apiVersion: v1
kind: Secret
metadata:
  name: git-credentials
data:
  id_rsa: BASE64ENCODED_PRIVATE_KEY
```

Here's a Python script to use to convert it to base64:

```
import base64

# Read the private key
with open('tektonkey', 'rb') as f:
    data = f.read()

# Encode to Base64
encoded_bytes = base64.b64encode(data)

# Convert to string
encoded_string = encoded_bytes.decode('

# Write to a file instead of printing
with open('tektonkey.b64', 'w') as f:
    f.write(encoded_string)

print("Base64 key saved to tektonkey.b6
```

Warning

Your Tekton Pipeline will fail to authenticate with GitHub if the private key is not stored correctly.

To avoid unnecessary debugging, we recommend using the Python script above. The script reads your private key, converts it to Base64, and saves the result to a file named `tektonkey.b64`. Copy the encoded string from this file and paste it into the Kubernetes manifest shown earlier.

If your pipeline still cannot authenticate with GitHub, consider the following troubleshooting tips:

1. **Update the deploy key in GitHub.**
   Whenever you generate a new key pair, you must create a new Deploy Key in GitHub. Previously registered public keys will not work with newly generated private keys.
2. **Replace the Secret in the cluster.**
   After creating a new key, delete the existing Secret in the cluster and apply a new one containing the updated private key.
3. **Verify the stored key in the cluster.**
   Use the command below to inspect the private key stored in the Secret. Ensure it exactly matches the key generated locally.

```
kubectl get secret git-credentials \
  -o jsonpath='{.data.id_rsa}' | base64
```

For the tasks we will use, this is the easiest way to configure our authentication. Tekton does support more modern
authentication approaches, but to keep the discussion easy to follow, we will use this approach.

Create the Kubernetes secret by using the `kubectl apply` command.

Warning

Do not commit Kubernetes Secret files to your repository. Secret manifests contain sensitive credentials in
base64-encoded form, which is easily reversible. Keep them only on your local machine.

This is separate from your Tekton pipeline YAML files (Tasks, Pipelines, PipelineRuns), which contain no credentials
and **should** be committed to Git — both as a backup and to make the transition to the CD task easier.

Now that we have our GitHub secret created, let's also authenticate our CI pipeline with our ACR.
To do this quickly, we have prepared a script for you to use:

Go to the "Access Keys" section of your ACR on the Azure portal, and copy the username and either of the passwords shown there.

```
USER_NAME=<acr_admin_username>
PASSWORD=<acr_admin_password>

echo -n $USER_NAME:$PASSWORD | base64 -

kubectl create secret generic docker-cr
```

Don't forget to replace <YOUR_ACR_NAME> with the name of your ACR.

Here's what the above command does:
We concatenated the USER_NAME and PASSWORD

variables together, with a colon between them, encoding them as base64 strings.
We then used `jq` to create a JSON with the following format:

```
{"auths":{"YOUR_ACR_NAME>.azurecr.io":{
```

Information

Why this specific format? This JSON structure is the standard configuration format used by the Docker CLI.

By storing it this way, we can mount this secret directly into our build containers (like Kaniko). Kaniko will read this file exactly as if it were a local Docker client, allowing it to authenticate with the registry
without any custom logic.

With authentication set up, we are ready to define the building blocks of our pipeline.

Information

Tip: Commit your Tekton YAML files (Tasks, Pipelines, PipelineRuns) to Git as you create them.

This creates a backup of your cluster configuration — saving you if you accidentally delete something — and will make the transition to the final Continuous Deployment (CD) task significantly easier.
Remember: YAML manifests are safe to commit. Kubernetes Secret files are not.

Subtask 3: Starting our Pipeline

To build our pipeline, we need to assemble three specific components:

1. **The Tasks:** We need manifests that define how to clone git repos and build Docker images.
2. **The Pipeline:** We need a manifest to wire these tasks together in the right order.
3. **The PipelineRun:** We need a manifest to actually trigger the execution.

With **Tekton**, like most **CI** tools, there is a place to grab existing **Task** definitions from, so you aren't

starting from scratch. There are two ways to use these definitions:

1. Use the Tekton CLI to install and deploy each task to your system.
2. Download the Task manifests and install them locally.

For this task, following best practices, we suggest **downloading the task manifests** and using them locally.
Tekton's pre-defined tasks are not heavily vetted, and so there are quite a few that do not specify the best way to
configure them for our use case. Downloading them locally also means that **we will always know exactly what we are
running**, which makes it straightforward to transition this repo to our CD solution in the next task.

Let's start by creating a `tekton` folder in the project root directory. We'll be organizing our CI-related files in this directory.
Next, let's get the two tasks that will handle the most complex part of the pipeline

> git cloning our repository and
> building and pushing our image to the ACR

We'll put these files in the `tekton` folder. CD into the tekton directory and run these two commands:

```
wget https://raw.githubusercontent.com/
wget https://raw.githubusercontent.com/
```

Let's look into both of these to get an idea of how **Task** files are structured. We'll start with [git-clone]:

Open `git-clone.yaml` and look at the spec section. You will see the core anatomy of a Tekton Task:

1. A description of what the task will do.
2. **Workspaces**: These are the inputs/outputs that require storage (like a PVC or a Secret).
   Examples: `output` (where the code goes), `ssh-directory` (where the keys are).

3. **Params**: These are configuration variables (strings/arrays).
   Examples: `url` (the git repo), `revision` (the branch).
4. **Results:** These are values the task returns to the pipeline.
   Example: The specific commit hash that was cloned (`commit`).
5. **Steps**: This is the most important part.
   Think of these exactly like Kubernetes `initContainers`. They are a list of containers that run sequentially.
   Step 1 finishes → Step 2 starts.
   They all share the same `Workspaces` (volume mounts), allowing Step 1 to download a file and Step 2 to read it.

While the YAML is verbose, the logic is straightforward.
It is a declarative manifest (like a Deployment) that describes a sequential process (like a script).
We define what we want (inputs, outputs, commands), and Tekton handles the execution.

The kaniko task is similarly easy to read:

Why Kaniko? Standard Docker builds require access to a Docker Daemon (the background process that manages containers).
However, modern Kubernetes clusters (including AKS) often use runtimes like `containerd` and do not expose a
Docker Daemon for security reasons.

Kaniko solves this by building container images entirely in userspace. It doesn't need a daemon, making it the standard,
secure way to build images inside a Kubernetes cluster.

With both of these tasks downloaded, use `kubectl apply -f` to register them with your cluster. This makes them
available to reference by name in any pipeline you define.

Subtask 4: A Simple Pipeline

We will build the first half of this pipeline together (cloning and building the LLM service).
Once you understand the pattern, you will complete the pipeline by adding the Login service and Testing steps yourself.

Information

The example above shows the structure, but you will need to fill in the correct parameter names and workspace mappings
based on the Task definitions you downloaded earlier.
Pay close attention to indentation — YAML is strict!

The best way to build a pipeline is to start by **reading** the task specifications, and looking for any and all **important parameters and workspaces** we will want to mount.

For **git-clone**, this will include:

url — our GitHub repository's SSH url.
ssh-directory — the workspace with our GitHub SSH information.
output — the directory that will store our cloned repository.

For **kaniko**, this will include:

source — the workspace with our source code.
dockerconfig — the workspace with our docker credentials secret.
IMAGE — the full tag of the image we want to build and push.
DOCKERFILE — the path, relative to the repository's root, that points to the Dockerfile we want to use.
CONTEXT — the path, relative to the repository's root, that points to the folder we want to build in (either `./django-authentication` or `./llm_server` in our case).

You do not need to make any changes for now.

With all of this information, let's now build our pipeline.
Create a file, `pipeline.yaml` in the **tekton** directory.

We know that we will need our **git-clone** task before our **kaniko** task, since without the cloned repository there is no source code to build from.

A simple **Pipeline** specification looks like the following:

```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: pipeline
spec:
  params:
    - name: mycoolparam
  workspaces:
    - name: source
  tasks:
    - name: gen-code
      taskRef:
        name: gen-code # gen-code expec
      workspaces:
        - name: source # <- mapping wor
    - name: commit
      taskRef:
        name: commit # commit expects a
      workspaces:
        - name: source # <- mapping wor
      runAfter:
        - gen-code
```

Here:

spec.params is a list of all parameters that someone needs to specify for our pipeline to be functional. In our case, this will be the **repo-url** and **image-reference** (our ACR's domain name).
spec.workspaces is a list of our workspaces. We will need three — one to **store our cloned code**, one to **hold our docker credentials**, and one to **hold our git credentials**.
spec.tasks is a list of tasks in our pipeline. In this case, there are two tasks, **gen-code** and **commit**. Here, **gen-code** must run before **commit** because `commit` lists `gen-code` in its **runAfter** field. At a minimum, you must specify all

parameters and **workspaces** each task will use, as well as **taskRef**, which contains the **name** of the task you want to use (in our case **kaniko** and **git-clone**).

We'll start by defining our pipeline's name and metadata:

```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: p5ci
  namespace: default
```

> **Information**
>
> The pipeline name and task names need to match exactly what we specify — grading depends on them.

Then, we move onto the spec.

Let's use the following parameters:

```
spec:
  params:
  - name: repo-url
    type: string
  - name: image-reference
    type: string
```

Here, `repo-url` refers to our repository's **SSH url**, while `image-reference` refers to our ACR's domain name, like `<YOUR_ACR_NAME>.azurecr.io`.

Let's also use the following workspaces:

```
workspaces:
- name: shared-data
- name: git-credentials
- name: docker-credentials
```

As you can see, **we don't specify the values of any parameter or workspace in a Pipeline definition**.

Instead, you specify them in a **PipelineRun** definition, which we will get to next.

Once that's done, we will add our first task — our **git-clone** task.

```
- name: fetch-source
  taskRef:
    name: git-clone
  workspaces:
  - name: output
    workspace: shared-data
  - name: ssh-directory
    workspace: git-credentials
  params:
  - name: url
    value: $(params.repo-url)
```

Here, **output** links to the **shared-data** workspace we defined previously, and **ssh-directory** does the same to the **git-credentials** workspace.

You will also note that, to use the **repo-url** parameter, we need to enclose it in `$()`. This is because Tekton acts like Bash and substitutes the value of the parameter at runtime, rather than treating it as a literal string.

Next, let's add our build-wheel task. This builds a wheel file from our source code. This task must run after the fetch-source task.

```
- name: build-wheel
  runAfter: ["fetch-source"]
  taskSpec:
    workspaces:
      - name: shared-data
    steps:
      - name: build
        image: python:3.13
        workingDir: $(workspaces.shar
        script: |
          pip install poetry
          poetry build -f wheel
```

Next, let's add our **build-push-llm** task. We will utilize this feature with our **kaniko** build task for our **llm** service:

```
  - name: build-push-llm
    runAfter: ["build-wheel"]
    taskRef:
      name: kaniko
    workspaces:
    - name: source
      workspace: shared-data
    - name: dockerconfig
      workspace: docker-credentials
    params:
    - name: IMAGE
      value: $(params.image-reference)/
    - name: DOCKERFILE
      value: ./llm_server/docker/Docker
    - name: CONTEXT
      value: ./llm_server
```

As you can see, our **IMAGE** parameter contains a path which specifies the registry, the image name (`llm`), as well as `$(tasks.fetch-source.results.committer-date)-$(tasks.fetch-source.results.commit)`. All this last part does is **read the result values from our fetch-source task**. This enables us to **automatically tag our image** with the Linux timestamp followed by a dash and the **commit hash**.

Information

You may wonder why we are not using the `latest` tag, as in previous projects. Tagging with the commit timestamp and hash lets us know **exactly when and from which commit** this image was built — invaluable when auditing a cluster. It also makes our **CD process** more transparent, since we can tell at a glance whether a deployment has caught up to a given commit.

To apply the above pipeline configuration to the cluster, run the following command:

```
kubectl apply -f pipeline.yaml
```

Once you have your pipeline file written and **applied** to the cluster, we can do our first test run with a `pipelinerun.yaml` file like the following:

```
apiVersion: tekton.dev/v1beta1
kind: PipelineRun
metadata:
  generateName: p5ci-
spec:
  pipelineRef:
    name: p5ci
  podTemplate:
    securityContext:
      fsGroup: 65532
  workspaces:
  - name: shared-data
    volumeClaimTemplate:
      spec:
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 3Gi
  - name: git-credentials
    secret:
      secretName: git-credentials
  - name: docker-credentials
    secret:
      secretName: docker-credentials
  params:
  - name: repo-url
    value: git@github.com:SAIL-Classroo
  - name: image-reference
    value: <YOUR_ACR_NAME>.azurecr.io
```

Warning

If you're not using the `SAIL-Classroom` organization, you should check from the repository on GitHub, the correct repo url to use for SSH.

A PipelineRun is safe to commit to Git — it contains no credentials, only workspace bindings and parameter values.

Some highlights:

The **generateName** metadata automatically generates a unique name for each run. This is helpful when iterating, as you do not need to manually rename each run.
We use a **securityContext** that lets us run as a normal user, ensuring the build behaves consistently.
We allocate 3 gigabytes of storage for the cloned repository workspace, which is more than sufficient.

Edit the above PipelineRun with your repository details, then run:

```
kubectl create -f pipelinerun.yaml
tkn pipeline logs -f -L
```

We use `kubectl create` (not `kubectl apply`) here because `generateName` requires a new resource to be created each time. Using `apply` would attempt to update an existing resource by name, which would fail since the name is auto-generated.

The `tkn pipeline logs -f -L` command is a handy shortcut that:

1. Finds the most recently started pipeline run.
2. Streams its logs so you can watch them in real time.

Pipeline Optimization

You should see that your pipeline takes way too long to run.
As a Cloud Practitioner, how can you modify the pipeline to make things faster?
Think about what steps you can cache to accomplish this.
You should get this right before proceeding to the next subtask.

Interim Validation

You are done with this subtask if, after triggering the PipelineRun, you see the following at the end of your logs:

```
INFO[0227] Pushed image to 1 destinatio

<YOUR_ACR_NAME>.azurecr.io/llm:17136922
```

Alternatively, you should see this in your Tekton Dashboard:

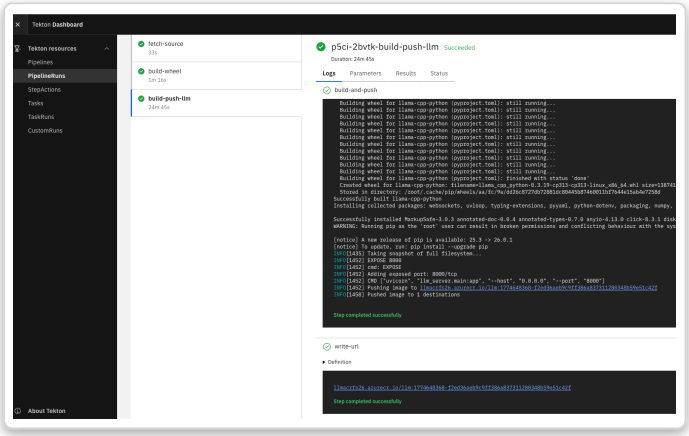


Figure: Tekton Pipeline Run

> **Information**
>
> If you run into trouble authenticating, feel free to use any other form of authentication provided in the documentation.

This confirms your first pipeline is working end-to-end. We can now move on to the remaining subtasks.

You will notice that the remaining subtasks contain less scaffolding — this is intentional. Your goal is to extend your pipeline incrementally, verifying that the cluster is working as expected at each step.

Subtask 5: Building the Login Service

First, update your `pipeline.yaml` to include a second **kaniko** task for the Login service.

> The task name should be: `build-push-django`.
> This should run after `fetch-source`.

The image name should be: $(params.image-reference)/login:$(tasks.fetch-source.results.committer-date)-$(tasks.fetch-source.results.commit)

Information

Hint: This can be done by copying the `build-push-llm` task block in your pipeline and editing the image name, Dockerfile path, and context. You will **not** need to edit the `pipelinerun.yaml` file.

Interim Validation

Start a pipeline run and confirm that the Login service builds successfully. Now that our pipeline is becoming more verbose, use the following command to get the latest run's status summary:

```
tkn pipelinerun describe -L
```

When you succeed, you should see output like the following:

```
Taskruns

 NAME                              TASK
 • p5ci-v22fr-build-push-django    build
 • p5ci-v22fr-build-push-llm       build
 • p5ci-v22fr-build-wheel          build
 • p5ci-v22fr-fetch-source         fetch
```

Subtask 6: Docker Security Analysis with Trivy

Now that we have a system that can automatically build and push our images, let's add a task to scan our **Docker images** for security vulnerabilities, which we can then decide to address at our convenience.

To do so, we will use `Trivy`, a tool purpose-built for this task. It will produce a list of known vulnerabilities, giving us a clear picture of our containers' current security posture.

If we look at https://artifacthub.io/packages/tekton-task/tekton-catalog-tasks/trivy-scanner, we will see that there is an existing Trivy task `manifest` we can use, with the following specification:

```
---
---
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: trivy-scanner
  labels:
    app.kubernetes.io/version: "0.2"
  annotations:
    tekton.dev/pipelines.minVersion: "0
    tekton.dev/categories: Security
    tekton.dev/tags: CLI, trivy
    tekton.dev/displayName: "trivy scan
    tekton.dev/platforms: "linux/amd64,
spec:
  description: >-
    Trivy is a simple and comprehensive
    vulnerabilities in container images
    ,and Git repositories, as well as f

    This task can be used to scan for v
    in stand alone mode.
  workspaces:
    - name: manifest-dir
  params:
    - name: ARGS
      description: The Arguments to be
      type: array
    - name: TRIVY_IMAGE
      default: docker.io/aquasec/trivy@
      description: Trivy scanner image
    - name: IMAGE_PATH
      description: Image or Path to be
      type: string
    - name: AIR_GAPPED_ENABLED
      default: "false"
      description: a flag enabling Air-
      type: string
  steps:
```

```
    - name: trivy-scan
      image: $(params.TRIVY_IMAGE)
      workingDir: $(workspaces.manifest
      script: |
        #!/usr/bin/env sh
          cmd="trivy $* "
          if [ "$(params.AIR_GAPPED_ENA
            echo "Air-Gapped mode enabl
            TRIVY_TEMP_DIR=$(mktemp -d)
            trivy --cache-dir "$TRIVY_T
            tar -cf ./db.tar.gz -C "$TR
            rm -rf "$TRIVY_TEMP_DIR"
            mkdir -p "$HOME"/.cache/tri
            tar xvf ./db.tar.gz -C "$HO
            cmd="${cmd}--skip-update "
          fi
          cmd="${cmd}$(params.IMAGE_PAT
          echo "Running trivy task with
          echo "$cmd"
          eval "$cmd"
      args:
        - "$(params.ARGS)"
```

In the tekton directory, create a file `trivy.yaml`. Copy and paste the above contents into the **trivy.yaml** file.

Edit the above task so that:

We add the workspace `docker-credentials` to our Trivy task definition. There's already the workspace, **manifest-dir** defined. Just add the below configuration under workspaces:

```
  - name: docker-credentials
    mountPath: /docker
```

Under the `trivy-scan` step, add the following to set the environment variable accordingly:

```
env:
- name: DOCKER_CONFIG
  value: /docker
```

Next, you will apply the above manifest to your cluster using the `kubectl apply -f trivy.yaml` command.

Now, how do you use this Trivy task?
You call it in the pipeline.yaml file. Here's an example usage for scanning your llm container image:

```
- name: scan-dep-llm
    taskRef:
      name: trivy-scanner
    workspaces:
    - name: manifest-dir
      workspace: shared-data
    - name: docker-credentials
      workspace: docker-credentials
    params:
    - name: IMAGE_PATH
      value: "$(tasks.build-push-llm.re
    - name: ARGS
      value: [ "image", "--severity", "
```

Your tasks are straightforward:

1. Add the above scan-dep-llm step to your `pipeline.yaml`. Add a condition so it runs only after the llm container is built and pushed to the ACR.
2. Do same for the `django service`. This step should be named `scan-dep-django` and should run only after the image is built and pushed to the ACR. Don't forget to udate the image path.

Interim Validation

After applying your updated pipeline and triggering a new PipelineRun, confirm your last run's describe output looks like the following:

```
tkn pipelinerun describe -L
```

```
📦 Taskruns

 NAME                                TASK
 · p5ci-628nq-scan-dep-llm           scan-
```

```
• p5ci-628nq-scan-dep-django     scan-
• p5ci-628nq-build-push-llm      build
• p5ci-628nq-build-push-django   build
• p5ci-628nq-build-wheel         build
• p5ci-628nq-fetch-source        fetch
```

Subtask 7: Adding Testing

Lastly, we will add a task to run our test suites inside of Tekton. Placing tests in the pipeline is how we create a **testing gate** — a step that blocks a build from proceeding if the tests fail, ensuring we never push broken images to our registry.

This time, there is no existing task for this within Tekton Hub. We will write our task from scratch. Given that you have already read and edited a task definition, this is very achievable.

Use the following as a starting point, using the name `run-tests` for this task.

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: run-tests
  namespace: default
  labels:
    app.kubernetes.io/version: "0.1"
  annotations:
    tekton.dev/pipelines.minVersion: "0
    tekton.dev/categories: Testing
    tekton.dev/displayName: "Run Tests"
spec:
  description: >-
    A simple task that allows for easy
  workspaces:
    - name: source
  steps:
    - name: run-fastapi-tests
      image: docker.io/library/python:3
      workingDir: $(workspaces.source.p
      script: |
        set -e
        apt-get update && apt-get insta
```

```
        pip install poetry
        poetry install
        poetry run pytest
    - name: run-django-tests
      image: docker.io/library/python:3
      workingDir: <PATH_TO_DJANGO_DIREC
      script: |
        set -e
        <STATEMENT_TO_INSTALL_DEPENDENC
        <STATEMENT_TO_RUN_TESTS>
```

1. In the tekton directory, copy the above YAML configuration into the file named `tests.yaml`.
2. The **run-fastapi-tests** have been implemented. Look on that to complete the **run-django-tests**

Pipeline update: Remember to update your `pipeline.yaml`. You need to place this `run-tests` task after `fetch-source` but before `build-push-*`.

Use `runAfter: ["fetch-source"]` for the `run-tests` task, and update both build-push-django task to use `runAfter: ["run-tests"]`. This ensures we never build or push an image from code that fails its tests.

> When iterating, it can be useful to clear out past runs using `tkn pipelinerun delete --all`. A large backlog of failed runs can slow down your cluster.
> Don't forget to apply the new manifest, `tests.yaml` to the cluster as well as the updated `pipeline.yaml` manifest.

Once you are done, you are ready to proceed to your submission.

Task 1 Final Validation

For this task, we will assess your pipeline by looking at the **most recent run**.

In particular, we will look for the following:

1. That all tasks completed successfully.
2. That the task names match exactly what is specified in this writeup.

3. That the logs for each task match the expected output.

Tekton Pipeline State

📦 `Taskruns`

```
NAME                                TASK
• p5ci-gswsn-scan-dep-django        scan-
• p5ci-gswsn-scan-dep-llm           scan-
• p5ci-gswsn-build-push-llm         build
• p5ci-gswsn-build-push-django      build
• p5ci-gswsn-run-tests              run-t
• p5ci-gswsn-build-wheel            build
• p5ci-gswsn-fetch-source           fetch
```

Directory Tree

```
llm_server/k8s/
├── deployment.yaml
├── model-pvc.yaml
└── service.yaml

1 directory, 3 files

tekton/
├── git-clone.yaml
├── kaniko.yaml
├── pipeline.yaml
├── pipelinerun.yaml
├── tests.yaml
└── trivy.yaml

1 directory, 6 files
```

Once you feel ready, please submit using the following:

```
./submitter -t task1
```

Reflection Questions

please copy these questions into the `answers/unit4answers.txt` file and answer them.

1. How did you optimize your CI pipeline to run fast?
2. Why is it necessary to optimize your CI pipelines for faster execution?
3. Discuss 2 reasons why vulnerability checks of your build images is critical.

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform